

ANGULAR UPLOAD COMPONENT TO SAVE FILE IN FIREBASE

© L. Hernández, 2024



INTRODUCTION

Before *SPA pattern (Single Page Applications)* emerged, traditional method for uploading files posted an entire *form*.

Form wrote file content to *request stream* leaving browser to appear hanging.

INTRODUCTION

Browser capabilities advanced in rise of *XHR* (*XMLHttpRequests*) and *AJAX* (*Asynchronous JS and XML*) allowing any *HTTP* request to be made between browser (client side code) and server.

These advancements opened ability to send *asynchronous requests* and observe progress of request enhancing user experience.

INTRODUCTION

Assembling some Angular Material components and using Firebase Storage, UI can present ability to start, pause, cancel, and display progress of a file upload.

CREATE FILE UPLOAD INTERFACE AND SERVICE CONTRACT

FileUpload interface contains information about file chosen and status of HTTP request.

CREATE FILE UPLOAD INTERFACE AND SERVICE CONTRACT

```
export interface FileUpload {  
  id: string;  
  name: string;  
  fileData: File,  
  size: number;  
  percentageUploaded: number;  
  status: FileUploadStatus;  
  contentType: string;  
  path?: string;  
  errorMessage?: string;  
}
```

A decorative footer consisting of a series of parallel diagonal lines in a light blue-grey color, spanning the entire width of the page.

CREATE FILE UPLOAD INTERFACE AND SERVICE CONTRACT

FileUploadStatus enumerates status of file upload:
QUEUED, PAUSED, CANCELLED, IN_PROGRESS,
COMPLETE, and ERROR.

CREATE FILE UPLOAD INTERFACE AND SERVICE CONTRACT

```
export enum FileUploadStatus {  
  QUEUED = 'queued',  
  PAUSED = 'paused',  
  CANCELLED = 'cancelled',  
  IN_PROGRESS = 'in progress',  
  COMPLETE = 'complete',  
  ERROR = 'error'  
}
```

CREATE FILE UPLOAD INTERFACE AND SERVICE CONTRACT

With different **UploadService** implementations, swapping dependency eases task of changing a specific reference as we'll inject **IFileuploadService** interface (or contract).

CREATE FILE UPLOAD INTERFACE AND SERVICE CONTRACT

```
import { Observable } from "rxjs";  
import { FileUpload } from "../models";  
  
export interface IFileUploadService {  
  readonly fileUploads$: Observable<Record<string, FileUpload>>;  
  upload(fileUploadId: string): void;  
  remove(fileUploadId: string): void;  
  addFiles(fileList: FileList, path: string): void;  
}
```



CREATE FIREBASE UPLOAD SERVICE

FirestoreUploadService handles task of uploading files to Firebase.

AngularFirestore class is injected into constructor of this service and it finally handles upload into Firebase.

CREATE FIREBASE UPLOAD SERVICE

upload method finds file by its **id** in **queuedFiles** property.

Using **path** and **fileData**, **fileStorage** creates an **AngularFireTask** by calling **upload** method.

Generated task appends to **tasks** property, and **fileUpload** status updates to **IN_PROGRESS**.

CREATE FIREBASE UPLOAD SERVICE

task.snapshotChanges is an observable that emits changes in state of uploading file.

map operator transforms value emitted to a **FileUpload** type.

Once observable completes, **finalize** operator removes current task from **tasks** property.

As values are emitted, **queuedFiles\$** property updates **fileUpload** in it's dictionary.

```
import { Injectable } from '@angular/core';
import { AngularFireStorage,
  AngularFireUploadTask } from '@angular/fire/compat/storage';
import { FileUpload, FileUploadStatus,
  IFileUploadService } from '@ngserveio/ui-domain';
import { removeProperty, notEmpty } from '@ngserveio/utilities';
import firebase from 'firebase/compat/app';
import { BehaviorSubject, catchError,
  filter, finalize, map, Observable, of } from 'rxjs';
import { v4 as uuid } from 'uuid';
```

```
const {
  CANCELED,
  ERROR,
  PAUSED,
  RUNNING,
  SUCCESS
} = firebase.storage.TaskState;
...
```

...

```
const fileState: Record<firebase.storage.TaskState, FileUploadStatus> = {  
  [CANCELED]: FileUploadStatus.CANCELLED,  
  [ERROR]: FileUploadStatus.ERROR,  
  [PAUSED]: FileUploadStatus.PAUSED,  
  [RUNNING]: FileUploadStatus.IN_PROGRESS,  
  [SUCCESS]: FileUploadStatus.COMPLETE  
}
```

```
@Injectable({ providedIn: 'root' })
```

```
export class FirebaseFileUploadService implements IFileUploadService {  
  private tasks: Record<string, AngularFireUploadTask> = { };  
  private queuedFiles$ = new BehaviorSubject<Record<string, FileUpload>>({ });  
  
  public readonly fileUploads$:  
    Observable<Record<string, FileUpload>> = this.queuedFiles$.asObservable();  
  
  constructor(protected fileStorage: AngularFireStorage) { }  
}
```

...

...

```
public addFiles(fileList: FileList, path: string): void {  
    const fileUploads: Record<string, FileUpload> = {};
```

```
    for (let i = 0; i < fileList.length; i++) {  
        const file = fileList[i];  
        const id = uuid();
```

```
        fileUploads[id] = {  
            id,  
            name: file.name.toLowerCase(),  
            contentType: file.type,  
            status: FileUploadStatus.QUEUED,  
            percentageUploaded: 0,  
            fileData: file,  
            path  
        } as FileUpload;  
    }
```

...

...

```
public addFiles(fileList: FileList, path: string): void {  
    const fileUploads: Record<string, FileUpload> = {};
```

...

```
    this.queuedFiles$.next({  
        ...this.queuedFiles$.value,  
        ...fileUploads  
    });  
}
```

...

...

```
public upload(fileUploadId: string): void {  
    const fileUpload = this.queuedFiles$.value[fileUploadId];  
    const task = this.fileStorage.upload(  
        `${fileUpload.path}/${fileUpload.name}`,  
        fileUpload.fileData  
    )  
    this.tasks = {  
        ...this.tasks,  
        [fileUploadId]: task  
    };  
    this.queuedFiles$.next({  
        ...this.queuedFiles$.value,  
        [fileUpload.id]: {  
            ...fileUpload,  
            status: FileUploadStatus.IN_PROGRESS  
        }  
    });  
    ...  
}
```

...

```
task.snapshotChanges().pipe(  
  filter(notEmpty),  
  map(({ bytesTransferred, totalBytes, state }) => {  
    const percentageUploaded =  
      Math.round((bytesTransferred / totalBytes) * 100);  
  
    return {  
      ...fileUpload,  
      status: fileState[state],  
      percentageUploaded  
    } as FileUpload;  
  }),  
  catchError(() => {  
    return of({  
      ...fileUpload,  
      status: FileUploadStatus.ERROR  
    });  
  }),  
  ...
```

...

task.snapshotChanges().pipe(

...

finalize() => {

// remove property from object of tasks

this.tasks = removeProperty(this.tasks, fileUpload.id);

})

).subscribe(fileUpload => {

this.queuedFiles\$.next({

...this.queuedFiles\$.value,

[fileUpload.id]: fileUpload

});

})

}

...

...

```
public remove(fileUploadId: string): void {  
  this.queuedFiles$.next({  
    ...removeProperty(this.queuedFiles$.value, fileUploadId)  
  });  
};
```

```
const task = this.tasks[fileUploadId];
```

```
if (task) {  
  // cancel task remove it from record of tasks  
  task.cancel();  
  this.tasks = {  
    ...removeProperty(this.tasks, fileUploadId)  
  };  
}
```

```
}
```

```
}
```

```
}
```

IMPLEMENT FILE UPLOAD COMPONENTS

FileUploadComponent consumes **fileUpload @Input()**.

This component is responsible for displaying progress, file name, and percentage upload to user.

mat-progress-bar reads **fileUpload.percentageUploaded** to reflect upload progress.

IMPLEMENT FILE UPLOAD COMPONENTS

```
import { ChangeDetectionStrategy,  
  Component, Input } from '@angular/core';  
import { FileUpload } from '@ngserveio/ui-domain';  
  
@Component({  
  selector: 'ng-serve-file-upload',  
  templateUrl: './file-upload.component.html',  
  styleUrls: ['./file-upload.component.scss'],  
  changeDetection: ChangeDetectionStrategy.OnPush  
})  
export class FileUploadComponent {  
  @Input() public fileUpload: FileUpload;  
}
```



IMPLEMENT FILE UPLOAD COMPONENTS

```
<div class="flex flex-row justify-evenly">
  <label class="text-left w-1/2 text-gray-700">
    {{ fileUpload?.name }}
  </label>
  <label class="text-right w-1/2">
    {{ fileUpload?.percentageUploaded }}%
  </label>
</div>
<div class="flex flex-row">
  <mat-progress-bar mode="determinate"
    color="primary"
    [value]="fileUpload?.percentageUploaded || 0">
  </mat-progress-bar>
</div>
```

IMPLEMENT FILE UPLOAD COMPONENTS

FileUploadListComponent orchestrates user's actions having them choose files to upload, and start / remove uploads.

IMPLEMENT FILE UPLOAD COMPONENTS

In markup of **FileUploadListComponent**, a list of files selected displays each file for upload using **ng-serve-file-upload** component.

mat-icon-buttons empower user to start an upload or remove file from list

```
<div>
  <div *ngFor="let fileUpload of fileUploads$ | async | keyvalue">
    <ng-serve-file-upload [fileUpload]="fileUpload.value"></ng-serve-file-upload>
    <div class="flex flex-row justify-end">
      <button mat-icon-button
        color="accent" title="Remove" (click)="remove(fileUpload.value.id)"
        [disabled]="fileUpload.value.status === UPLOAD_STATUS.IN_PROGRESS">
        <mat-icon>delete</mat-icon>
      </button>
      <button mat-icon-button
        color="accent" title="Upload" (click)="uploadFile(fileUpload.value.id)"
        [disabled]="fileUpload.value.status !== UPLOAD_STATUS.QUEUED">
        <mat-icon>file_upload</mat-icon>
      </button>
    </div>
  </div>
</div>
</div>
...
```

...

```
<input (change)="filesSelected($event)"  
      type="file"  
      multiple  
      [attr.accept]="fileTypes"  
      class="file-upload-list__file-input"  
      #fileListUpload
```

```
/>
```

```
<button mat-button  
      class="rounded bg-green-500 text-white p-1 px-2 align-middle"  
      (click)="chooseFiles()">  
  <mat-icon>file_upload</mat-icon> Choose Files  
</button>
```

IMPLEMENT FILE UPLOAD COMPONENTS

FileUploadListComponent **fileListUpload** property exposes an **ElementRef** reference to **#fileListUpload file** input.

When "Choose Files" button fires *click event*, **chooseFiles** method invokes *click event* on **#fileListUpload file** input.

```
import { ChangeDetectionStrategy,  
  Component, ElementRef, Inject, Input, ViewChild } from '@angular/core';  
import { FileUploadStatus, IFileUploadService } from '@ngserveio/ui-domain';  
import { FileUploadInjectionToken } from './injection-keys';
```

```
@Component({  
  selector: 'ng-serve-file-upload-list',  
  templateUrl: './file-upload-list.component.html',  
  styleUrls: ['./file-upload-list.component.scss'],  
  changeDetection: ChangeDetectionStrategy.OnPush  
})
```

```
export class FileUploadListComponent {  
  @Input() public fileTypes: string = 'poop';  
  @Input() public path: string = 'asdf';
```

```
  @ViewChild('fileListUpload') public fileListUpload: ElementRef;
```

```
  ...
```

A decorative footer consisting of a series of parallel diagonal lines in a light blue-grey color, spanning the entire width of the image.

...

```
public fileUploads$ = this.fileUploadService.fileUploads$;  
public UPLOAD_STATUS = FileUploadStatus;
```

```
constructor(@Inject(FileUploadInjectionToken)  
    protected fileUploadService: IFileUploadService) { }
```

```
public chooseFiles(): void {  
    this.fileListUpload.nativeElement.click();  
}
```

```
public filesSelected(evt: Event): void {  
    const fileList: FileList = (evt?.target as HTMLInputElement)?.files;  
    if (!fileList) { return; }  
    this.fileUploadService.addFiles(fileList, 'test');  
}
```

...



...

```
public uploadFile(fileId: string): void {  
    this.fileUploadService.upload(fileId);  
}
```

```
public remove(fileId: string): void {  
    this.fileUploadService.remove(fileId);  
}  
}
```

RUN FILE UPLOAD APPLICATION

By default, requests made to Firebase Storage deny unauthorized requests so, for testing purposes, set **"allow read, write: if true;"**.

[Edit rules](#) Monitor rules



Guard your data with rules that define who has access to it and how it is structured

[View the docs](#)

```
1 rules_version = '2';
2 service firebase.storage {
3   match /b/{bucket}/o {
4     match /{allPaths=**} {
5       allow read, write: if request.auth != null;
6     }
7   }
8 }
9
```

RUN FILE UPLOAD APPLICATION

Go to <http://localhost:4200/forms/file-upload-list> and then we'll be able to add files, upload to Firebase, and remove files once completed.

